

Day 12

CDC:

Clock Domain Crossing (CDC) in VLSI is the process of transferring signals between circuits operating on different or asynchronous clocks, which can cause metastability, data corruption, and functional failures if not properly synchronized. Essential techniques include using two-flip-flop synchronizers for single-bit signals, asynchronous FIFOs for data buses, and handshake protocols, validated through CDC analysis tools.

Metastability in VLSI is an unstable, intermediate voltage state (neither logic 0 nor 1) occurring when flip-flops violate setup/hold times, often during asynchronous input sampling.

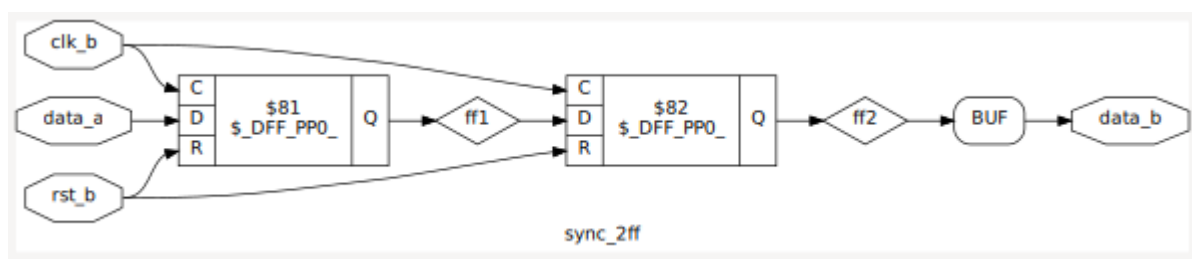
2 Flipflop synchronizer

Verilog Code:

```
module sync_2ff(
    input clk_b, // destination clock
    input rst_b,
    input data_a, // signal from domain A
    output data_b // synchronised output in domain B
);
    reg ff1, ff2;

    always @(posedge clk_b or posedge rst_b) begin
        if(rst_b) begin
            ff1 <= 0;
            ff2 <= 0;
        end
        else begin
            ff1 <= data_a; // FF1 may go metastable
            ff2 <= ff1;    // FF2 samples after metastability resolves
        end
    end

    assign data_b = ff2;
endmodule
```



2.26. Printing statistics.

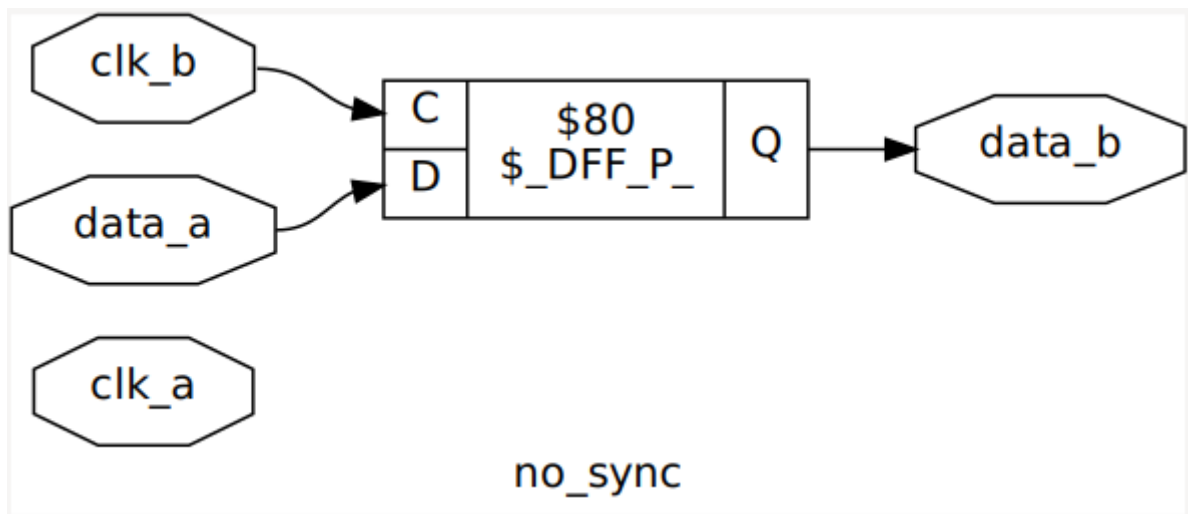
=== sync_2ff ===

Number of wires:	6
Number of wire bits:	6
Number of public wires:	6
Number of public wire bits:	6
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	2
\$_DFF_PPO_	2

Without Synchronizer:

Verilog code:

```
// WRONG — direct connection across clock domains
module no_sync(
    input clk_a, clk_b,
    input data_a,
    output reg data_b
);
    // This is a CDC violation — data_a driven by clk_a
    // directly sampled by clk_b
    always @(posedge clk_b)
        data_b <= data_a;
endmodule
```



6. Printing statistics.

=== no_sync ===

Number of wires:	4
Number of wire bits:	4
Number of public wires:	4
Number of public wire bits:	4
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	1
\$ _DFF_P_	1

One extra FF is the entire difference between a safe and an unsafe CDC crossing. The synthesis report looks almost identical — but one will cause random silicon failures and the other won't. This is why CDC can't be caught by looking at cell counts or schematics.

Multi bit CDC:

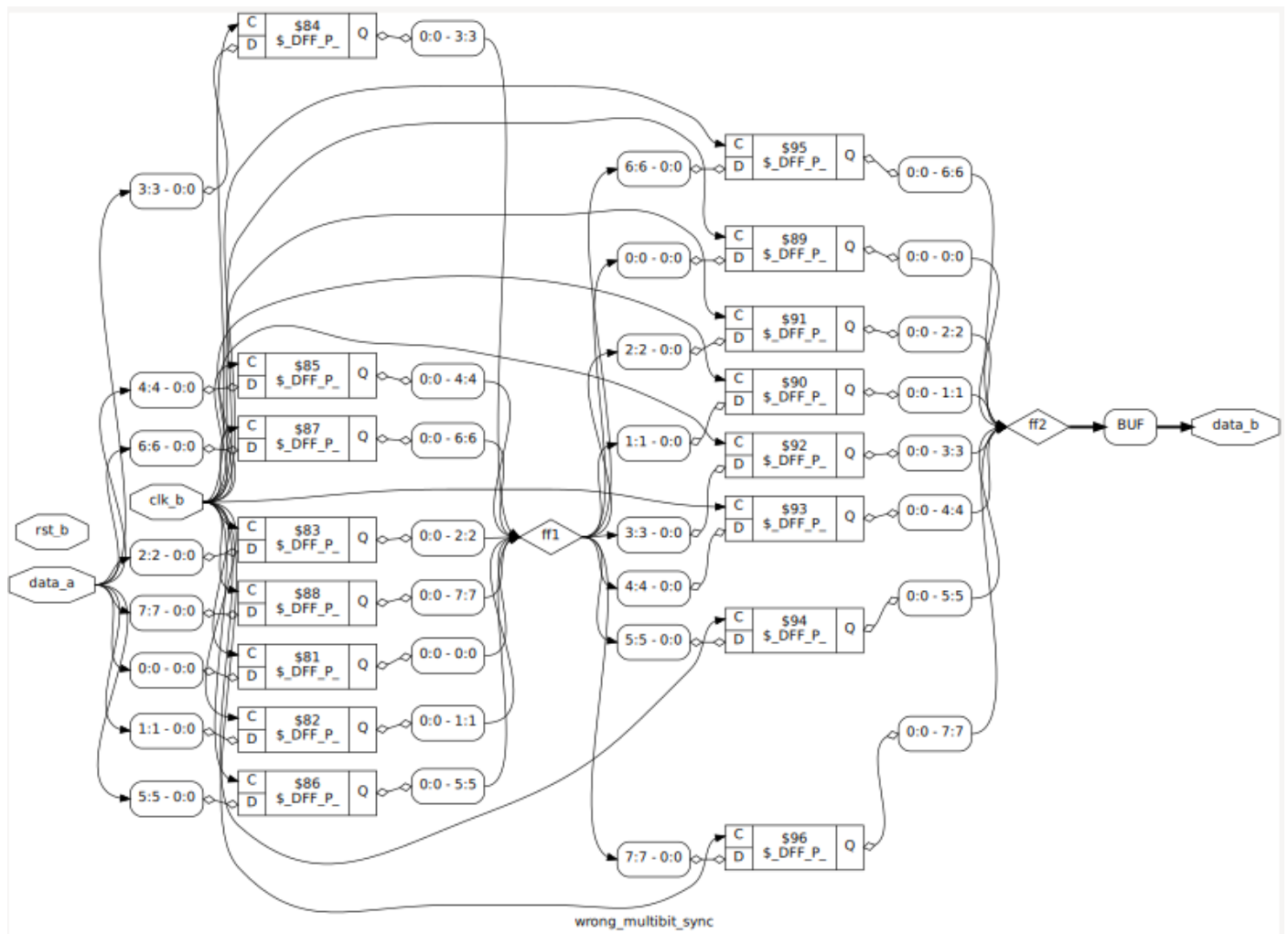
// WRONG — synchronising a multi-bit bus directly

```
module wrong_multibit_sync(
    input clk_b, rst_b,
    input [7:0] data_a,
    output reg [7:0] data_b
);
    reg [7:0] ff1, ff2;
    always @(posedge clk_b) begin
```

```

ff1 <= data_a; // Each bit may capture different values
ff2 <= ff1;    // STILL WRONG for multi-bit
end
assign data_b = ff2;
endmodule

```



3. Printing statistics.

=== wrong_multibit_sync ===

Number of wires:	6
Number of wire bits:	34
Number of public wires:	6
Number of public wire bits:	34
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	16
\$_DFF_P_	16

Gray code counter for CDC

```

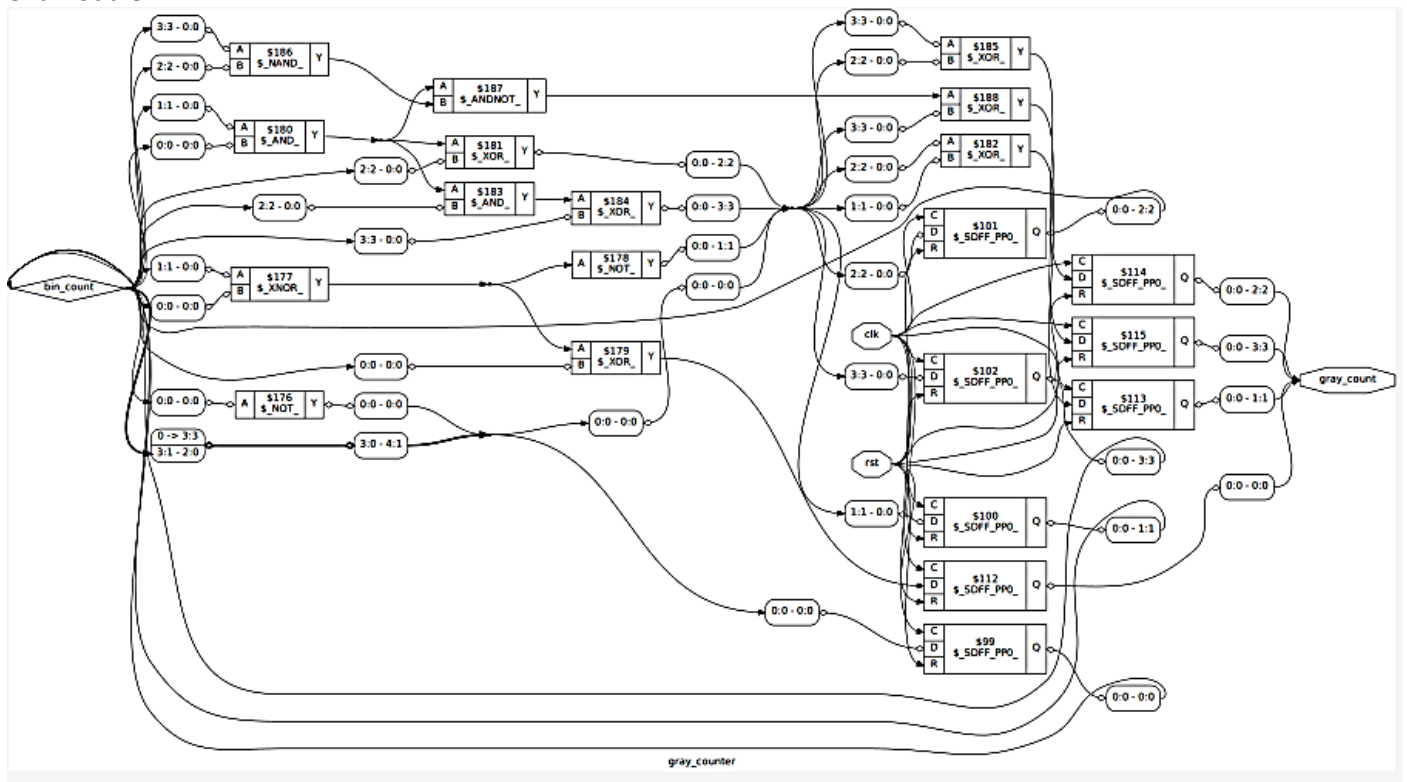
module gray_counter #(parameter WIDTH = 4)(
    input      clk, rst,
    output reg [WIDTH-1:0] gray_count
);
    reg [WIDTH-1:0] bin_count;

```

```

always @(posedge clk) begin
  if(rst) begin
    bin_count <= 0;
    gray_count <= 0;
  end
  else begin
    bin_count <= bin_count + 1;
    // Binary to Gray conversion
    gray_count <= (bin_count + 1) ^ ((bin_count + 1) >> 1);
  end
end
endmodule

```



3. Printing statistics.

=== gray_counter ===

Number of wires:	15
Number of wire bits:	29
Number of public wires:	4
Number of public wire bits:	10
Number of memories:	0
Number of memory bits:	0
Number of processes:	0
Number of cells:	21
\$_ANDNOT_	1
\$_AND_	2
\$_NAND_	1
\$_NOT_	2
\$_SDFF_PPO_	8
\$_XNOR_	1
\$_XOR_	6

The binary-to-gray conversion $(bin+1) \wedge ((bin+1) \gg 1)$ synthesised to 6 XOR gates — exactly one XOR per bit except the MSB. The schematic shows this clearly — a chain of XOR gates taking adjacent bits of the binary count.

Asynchronous FIFO:

Verilog code:

```
module async_fifo #(
    parameter WIDTH = 8,
    parameter DEPTH = 8
)(
    // Write domain
    input      wr_clk, wr_rst,
    input      wr_en,
    input [WIDTH-1:0] din,
    output      full,

    // Read domain
    input      rd_clk, rd_rst,
    input      rd_en,
    output [WIDTH-1:0] dout,
    output      empty
);
    localparam PTR_W = $clog2(DEPTH) + 1; // +1 for full/empty distinction

    // Memory
    reg [WIDTH-1:0] mem [0:DEPTH-1];

    // Write domain pointers (binary + gray)
    reg [PTR_W-1:0] wr_ptr_bin, wr_ptr_gray;

    // Read domain pointers (binary + gray)
    reg [PTR_W-1:0] rd_ptr_bin, rd_ptr_gray;

    // Synchronised pointers (gray code crossing domains)
    reg [PTR_W-1:0] wr_ptr_gray_sync1, wr_ptr_gray_sync2;
    reg [PTR_W-1:0] rd_ptr_gray_sync1, rd_ptr_gray_sync2;

    // — Write domain —————
    always @(posedge wr_clk or posedge wr_rst) begin
        if(wr_rst) begin
            wr_ptr_bin <= 0;
            wr_ptr_gray <= 0;
        end
        else if(wr_en && !full) begin
            mem[wr_ptr_bin[PTR_W-2:0]] <= din;
            wr_ptr_bin <= wr_ptr_bin + 1;
            wr_ptr_gray <= (wr_ptr_bin + 1) ^ ((wr_ptr_bin + 1) >> 1);
        end
    end

    // — Read domain —————
    assign dout = mem[rd_ptr_bin[PTR_W-2:0]];

    always @(posedge rd_clk or posedge rd_rst) begin
        if(rd_rst) begin
            rd_ptr_bin <= 0;
            rd_ptr_gray <= 0;
        end
        else if(rd_en && !empty) begin
            rd_ptr_bin <= rd_ptr_bin + 1;
            rd_ptr_gray <= (rd_ptr_bin + 1) ^ ((rd_ptr_bin + 1) >> 1);
        end
    end
end
```

```

    end
end

// — Sync wr_ptr to rd_clk domain (2-FF) ———
always @(posedge rd_clk or posedge rd_rst) begin
    if(rd_rst) begin
        wr_ptr_gray_sync1 <= 0;
        wr_ptr_gray_sync2 <= 0;
    end
    else begin
        wr_ptr_gray_sync1 <= wr_ptr_gray;
        wr_ptr_gray_sync2 <= wr_ptr_gray_sync1;
    end
end

// — Sync rd_ptr to wr_clk domain (2-FF) ———
always @(posedge wr_clk or posedge wr_rst) begin
    if(wr_rst) begin
        rd_ptr_gray_sync1 <= 0;
        rd_ptr_gray_sync2 <= 0;
    end
    else begin
        rd_ptr_gray_sync1 <= rd_ptr_gray;
        rd_ptr_gray_sync2 <= rd_ptr_gray_sync1;
    end
end

// — Full flag (in wr_clk domain) ———
// Full when wr_ptr and synced rd_ptr differ in
// top 2 bits but match in lower bits (Gray code full condition)
assign full = (wr_ptr_gray == {~rd_ptr_gray_sync2[PTR_W-1:PTR_W-2],
                                rd_ptr_gray_sync2[PTR_W-3:0]});

// — Empty flag (in rd_clk domain) ———
// Empty when rd_ptr == synced wr_ptr
assign empty = (rd_ptr_gray == wr_ptr_gray_sync2);

endmodule

```

Testbench:

```

`timescale 1ns/1ps
module tb;
    parameter WIDTH = 8;
    parameter DEPTH = 8;

    reg wr_clk, rd_clk;
    reg wr_rst, rd_rst;
    reg wr_en, rd_en;
    reg [WIDTH-1:0] din;
    wire [WIDTH-1:0] dout;
    wire full, empty;

    async_fifo #(WIDTH, DEPTH) uut(
        .wr_clk(wr_clk), .wr_rst(wr_rst),
        .wr_en(wr_en), .din(din), .full(full),
        .rd_clk(rd_clk), .rd_rst(rd_rst),
        .rd_en(rd_en), .dout(dout), .empty(empty)
    );
endmodule

```

```

);

// Different clock frequencies — this is the whole point
initial begin wr_clk = 0; forever #5 wr_clk = ~wr_clk; end // 100MHz
initial begin rd_clk = 0; forever #7 rd_clk = ~rd_clk; end // ~71MHz

initial begin
    $dumpfile("async_fifo.vcd");
    $dumpvars(0, tb);
end

initial begin
    wr_rst = 1; rd_rst = 1;
    wr_en = 0; rd_en = 0; din = 0;
    #30;
    wr_rst = 0; rd_rst = 0;
    #10;

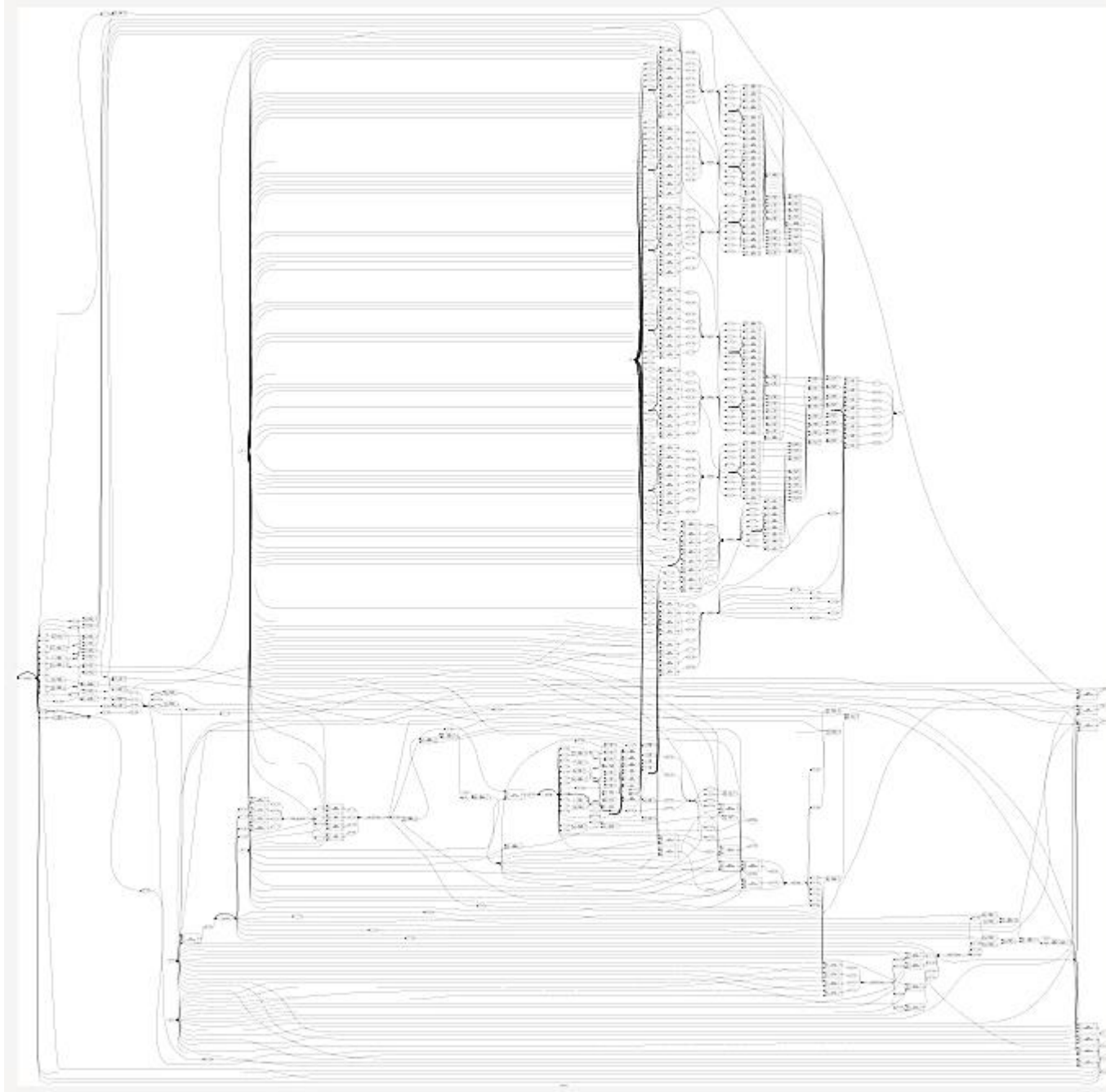
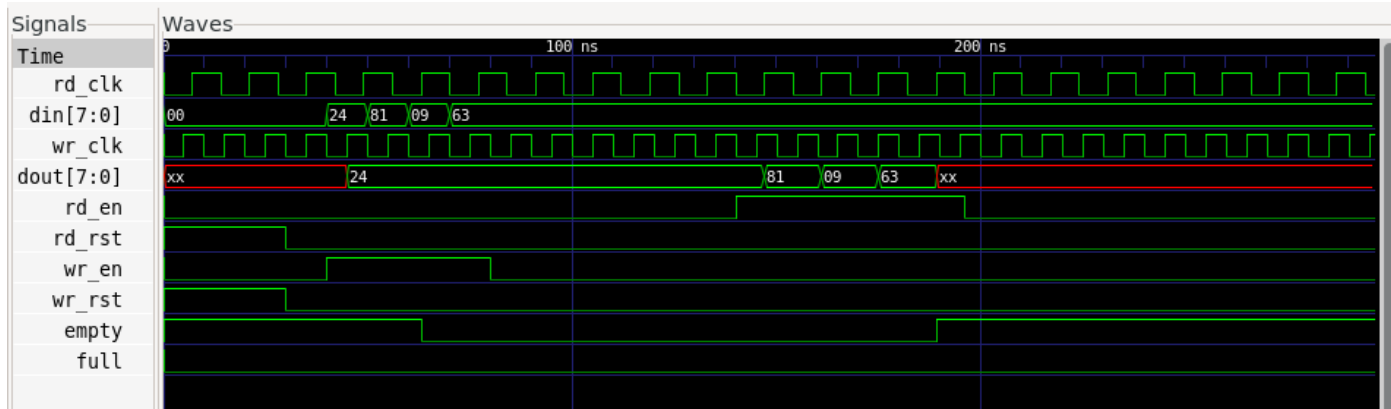
    // Write 4 bytes fast (write domain)
    repeat(4) begin
        @(negedge wr_clk);
        if(!full) begin
            wr_en = 1;
            din = $random;
            $display("Write: %h", din);
        end
    end
    @(negedge wr_clk); wr_en = 0;

    // Wait a bit — let read domain catch up
    #50;

    // Read all out (read domain, slower clock)
    repeat(4) begin
        @(negedge rd_clk);
        if(!empty) begin
            rd_en = 1;
            @(posedge rd_clk); #1;
            $display("Read: %h", dout);
        end
    end
    @(negedge rd_clk); rd_en = 0;

    #100; $finish;
end
endmodule

```



=== async_fifo ===

Number of wires:	205
Number of wire bits:	315
Number of public wires:	26
Number of public wire bits:	120
Number of memories:	0

Number of memory bits:	0
Number of processes:	0
Number of cells:	289
\$_ANDNOT_	77
\$_AND_	6
\$_DFFE_PPOP_	16
\$_DFFE_PP_	64
\$_DFF_PPO_	16
\$_MUX_	8
\$_NAND_	5
\$_NOT_	8
\$_ORNOT_	4
\$_OR_	65
\$_XNOR_	4
\$_XOR_	16

Notice dout appears later than din — that's the sync latency. The gray pointer had to cross through 2 rd_clk cycles before the read domain knew data was available. That delay is CDC working correctly.